

Política neuronal por aprendizaje por refuerzo profundo para fútbol robótico VSSS

Del simulador de entrenamiento al despliegue en FIRASim

Nikolaj Hansen — Ingeniería Civil Informática
Benjamín Reyes — Ingeniería Civil Informática
Eduardo Morales — Ingeniería Civil Informática

INF398 — Introducción al Aprendizaje Automático
Universidad Técnica Federico Santa María, primer semestre de 2026

Resumen

Este trabajo estudia si una política neuronal aprendida puede reemplazar parte de la lógica programada manualmente en fútbol robótico Very Small Size Soccer (VSSS). Se implementó un pipeline completo de entrenamiento, exportación y despliegue: una política Actor-Critic con perceptrones multicapa de dos capas ocultas de 256 unidades se entrena mediante Proximal Policy Optimization (PPO), recibe una observación normalizada de 52 variables y controla dos robots de campo mediante acciones continuas (v, ω) a 10 Hz. El entrenamiento se realizó primero en un simulador cinemático propio, con currículo y self-play, y posteriormente en rSoccer, cuya física de contacto es más realista. La política se exportó a ONNX y se integró al motor Rust del equipo para comandar FIRASim en tiempo real. En el simulador propio, el mejor modelo alcanzó un neto de +89 puntos porcentuales frente al baseline basado en reglas; sin embargo, su transferencia a FIRASim fue deficiente. Una corrida de $20,56 \times 10^6$ pasos en rSoccer obtuvo solo 18 % de goles y 76 % de empates por timeout en su propio entorno, además de evidenciar colapso de exploración. El aporte principal es tanto el sistema funcional de extremo a extremo como el diagnóstico experimental que separa sobreajuste al simulador, inestabilidad de PPO, diseño de recompensa —incluida la ausencia de incentivos de cooperación multiagente— y desalineación entre entrenamiento y despliegue. A partir de este análisis se implementaron regularización, calibración por identificación de sistemas, ruido de observación y una recompensa orientada al remate.

Índice

1. Introducción	3
2. Problema, datos y formulación	3
2.1. Un problema secuencial sin dataset estático	3
2.2. Contrato de observación y procesamiento del estado	4
2.3. Acciones y diseño de la recompensa	4

3. Solución propuesta	5
3.1. Política Actor-Critic y PPO	5
3.2. Currículo, self-play y simuladores	5
3.3. Arquitectura de extremo a extremo	6
4. Implementación	6
4.1. El lazo de entrenamiento (Python)	6
4.2. El despliegue (motor Rust)	7
4.3. Decisiones de diseño	7
5. Metodología experimental	8
5.1. Métricas y protocolo	8
5.2. Experimentos	8
6. Resultados	8
6.1. Aprendizaje inicial	8
6.2. Dominio en el simulador propio	9
6.3. rSoccer: rendimiento y colapso	9
7. Análisis crítico y comparación	10
7.1. Dos modos de falla diferentes	10
7.2. La cooperación no se premió y, por lo tanto, no emergió	10
7.3. Brechas entrenamiento-despliegue	11
7.4. Comparación con trabajos relacionados	11
8. Conclusiones y trabajo futuro	12
Disponibilidad y reproducibilidad	13

1. Introducción

VSSS es una categoría de fútbol robótico autónomo en la que dos equipos de tres robots diferenciales compiten en una cancha de aproximadamente 1,5 m por 1,3 m. Una cámara cenital entrega posiciones y orientaciones; a partir de ellas, el software debe estimar el estado del mundo, decidir una estrategia y generar comandos de movimiento. En el software de Sysmic Robotics esta decisión se realiza tradicionalmente mediante **RuleBasedCoach**, un conjunto de reglas y máquinas de estado con roles como atacante, apoyo y arquero.

Este enfoque es interpretable y funciona con pocos recursos, pero cada comportamiento nuevo exige diseñar y ajustar condiciones manuales. Además, una política rígida tiene poca capacidad para adaptarse a rivales o descubrir estrategias no previstas. El aprendizaje por refuerzo (RL) ofrece una alternativa: optimizar una política a partir de experiencia generada al jugar, sin disponer de etiquetas de “acción correcta” para cada estado. PPO resulta apropiado para acciones continuas y ha mostrado un compromiso favorable entre estabilidad y costo computacional [1].

La pregunta de investigación es: *¿Puede una política neuronal entrenada por deep RL aprender comportamiento efectivo en VSSS y conservarlo al desplegarse en el simulador utilizado por el motor real del equipo?* El objetivo general es entrenar, integrar y evaluar esa política sobre el software existente. Los objetivos específicos son:

1. construir un entorno que genere experiencia y permita un currículo de dificultad creciente;
2. definir un contrato estable de observación y acción entre Python y Rust;
3. comparar la política con oponentes heurísticos mediante tasa de goles, duración y estabilidad;
4. desplegar la red en FIRASim y analizar la brecha entre entrenamiento y ejecución.

La formulación evolucionó desde la entrega parcial: se mantuvo el dominio VSSS y la arquitectura Actor-Critic, pero se sustituyó el control por ruedas del prototipo 1v0 por acciones (v, ω) , se completó el despliegue ONNX y se incorporó rSoccer. Esta continuidad permite observar una progresión sistemática desde 1,5 millones de pasos y un 5 % de goles hasta decenas de millones de transiciones, self-play, despliegue real en el motor y una auditoría del fallo de transferencia.

2. Problema, datos y formulación

2.1. Un problema secuencial sin dataset estático

No se utiliza un dataset supervisado. Los datos se generan en línea al interactuar la política con el entorno. Cada decisión produce una transición

$$(s_t, a_t, r_t, s_{t+1}),$$

donde s_t es la observación, a_t la acción continua, r_t la recompensa y s_{t+1} el estado siguiente. PPO acumula rollouts, estima ventajas y actualiza los parámetros por minibatches. La Fase 1 produjo 1.507.328 transiciones (aproximadamente 3.300 episodios) y la principal corrida en rSoccer acumuló $20,56 \times 10^6$ pasos, cerca de 22 horas de CPU.

El problema puede modelarse como un proceso de decisión parcialmente observable: la política recibe una estimación del estado en un instante, pero no conoce parámetros físicos exactos, intenciones del rival ni errores futuros de percepción. Durante el entrenamiento se ejecutan ocho entornos en

paralelo. La física de rSoccer opera a 40 Hz y se agrupan cuatro ticks por acción, por lo que la red decide a 10 Hz, igual que en el despliegue.

2.2. Contrato de observación y procesamiento del estado

La observación es un vector fijo de 52 números (table 1). El mismo layout se implementa en `training/vsss_rl/observation.py` y `src/coach/observation.rs`; esta duplicación deliberada constituye una interfaz contractual entre entrenamiento y ejecución. La figure 1 resume el pipeline que transforma las magnitudes crudas del simulador (o de la visión, en despliegue) en ese vector normalizado.

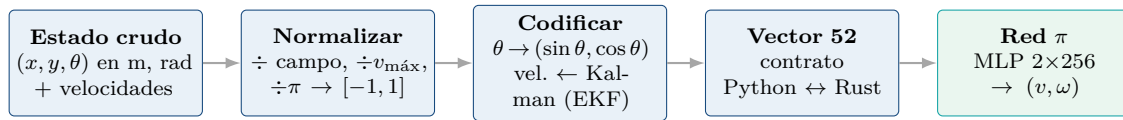


Figura 1: Pipeline de procesamiento del estado: de las magnitudes crudas del simulador/visión al vector normalizado de 52 entradas que consume la red.

Tabla 1: Composición y normalización de la observación.

Índices	Contenido	Cantidad	Normalización
[0, 4)	Pelota: x, y, v_x, v_y	4	$x/0,75, y/0,65, v/1,5$
[4, 28)	Tres robots propios: $x, y, v_x, v_y, \sin \theta, \cos \theta, \omega, active$	24	posición, velocidad y ω/π
[28, 52)	Tres robots rivales con el mismo formato	24	igual al bloque anterior

La orientación se representa mediante $(\sin \theta, \cos \theta)$ para eliminar la discontinuidad en $\pm\pi$. Los robots no visibles se rellenan con ceros y $active=0$. En simulación, las posiciones se perturban con ruido gaussiano medido ($\sigma_x = 1,854$ mm, $\sigma_y = 1,679$ mm y $\sigma_\theta = 0,031$ rad) y las velocidades se calculan por diferencia finita sobre 0,1 s; así se aproxima el ruido y retardo de las estimaciones que el filtro de Kalman entrega en despliegue.

2.3. Acciones y diseño de la recompensa

El actor entrega cuatro valores acotados, (v, ω) para cada uno de los dos robots de campo. Se desnormalizan con $v_{\text{máx}} = 1,2$ m/s y $\omega_{\text{máx}} = 3$ rad/s. El arquero se mantiene basado en reglas para acotar el problema experimental.

Por qué shaping. Con recompensa escasa (solo goles), una política inicial casi nunca anota: la señal es cero en la enorme mayoría de los pasos y el gradiente carece de información útil. El *reward shaping* agrega recompensas intermedias que marcan un camino hacia el gol. El diseño se organiza por intención (table 2) y su forma general es

$$r = 10 \cdot \text{gol} + w_r \cdot \text{remate} + \text{progreso} + \text{cercanía} - \text{faltas} - 0,01 \cdot \text{tiempo}.$$

Tabla 2: Términos de la recompensa, agrupados por intención.

Grupo	Término (valor)	Rol
Marcar	gol +10 / en contra -10	eventos terminales; el único objetivo real
Atacar	velocidad de la pelota hacia el arco (dominante)	premia el remate; peso guiado por Brandão et al. [2]
	progreso al arco + cercanía a la pelota (nivel)	guía sostenida hacia el balón
Reglas	faltas de área -1, <i>Free Ball</i> -0,5, quietud	árbitro interno estilo reglamento LARC 2026
Tiempo	-0,01 por paso	penaliza demoras

Nivel vs. gradiente (una lección de diseño). La versión inicial premiaba el *cambio* de distancia robot-pelota. Este término se telescopa: acercarse entrega recompensa una vez, pero alejarse la devuelve, de modo que la suma a lo largo del episodio tiende a cero y no compensa sostenidamente el riesgo de disputar la pelota. En la práctica esto indujo una política que *evitaba* el balón (section 6.3). La versión final usa la cercanía como *nivel* (recompensa constante mientras el robot esté sobre la pelota) y hace *dominante* la velocidad de la pelota hacia el arco, siguiendo el diseño de recompensas de Brandão et al. [2]. Las reglas del torneo se modelan como un árbitro interno que penaliza faltas y corta jugadas estancadas, en línea con el “árbitro virtual” de dicho trabajo.

3. Solución propuesta

3.1. Política Actor-Critic y PPO

La política utiliza dos torres MLP independientes que comparten la observación, pero no parámetros. Cada torre contiene capas $52 \rightarrow 256 \rightarrow 256$ con activación *tanh* y se implementa explícitamente como subclase de `torch.nn.Module`, aplicando los patrones estándar de perceptrón multicapa y retropropagación descritos en textos de referencia [9, 10]. El actor produce la media $\mu_\theta(s) \in \mathbb{R}^4$ de una gaussiana diagonal; $\log \sigma$ es un parámetro global aprendible. El crítico estima $V_\psi(s)$. La arquitectura tiene aproximadamente 159.500 parámetros y sigue la escala usada en trabajos VSSS y en rSoccer [2, 3].

PPO maximiza un objetivo sustituto recortado y entrena simultáneamente el crítico:

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1)$$

$$L_{\text{total}} = -L^{\text{clip}} + c_v L_{\text{valor}} - c_e H(\pi_\theta). \quad (2)$$

Aquí $\hat{A}_t$ es la ventaja estimada por GAE, $r_t(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$ es el cociente de probabilidades de la acción tomada, L_{valor} es el error cuadrático medio entre $V_\psi(s)$ y el retorno, y H es la entropía de la política (bonus de exploración). Se emplea Stable-Baselines3 (SB3) [7] con Adam [8], $\gamma = 0,99$, $\lambda_{\text{GAE}} = 0,95$, *batch* 256, *learning rate* 10^{-4} y *clip range* 0,2. La red y sus dimensiones son explícitas en PyTorch; SB3 aporta la implementación auditada del optimizador PPO, mientras que la observación, la acción y la recompensa —donde reside el diseño propio— se definen en el entorno.

3.2. Currículo, self-play y simuladores

El simulador propio permite iterar rápidamente y controlar la física. El currículo aumenta la complejidad de 1v0 a 1v1, 2v1, configuraciones mixtas y self-play contra snapshots. Mantener parte

del pool de oponentes fijo evita que una política parezca mejorar solo porque su rival cambia al mismo tiempo. Este stack consiguió el mejor resultado interno, pero su contacto de pelota simplificado permitió sobreajuste.

rSoccer ofrece una física ODE más realista y un entorno VSS diseñado para RL [3]. Se construyó un wrapper que conserva exactamente las 52 entradas y las acciones (v, ω) . La primera corrida larga no incorporó self-play, currículo, monitor ni regularización suficiente; sus fallos motivaron una segunda iteración con ruido de percepción, ejecutor alineado, árbitro interno y reward revisado.

3.3. Arquitectura de extremo a extremo

La [figure 2](#) resume el flujo. El actor determinista se exporta a ONNX; **RLCoach** lo ejecuta mediante **tract-onnx**. La salida se convierte en el skill **DirectVel**, evitando el planificador UVF/PID utilizado por las decisiones de alto nivel. Esta decisión reduce una fuente de discrepancia: la red entrena y se despliega sobre el mismo significado de acción.



Figura 2: Pipeline implementado desde la generación de experiencia hasta el despliegue. El ciclo de la izquierda (obs. $\rightarrow$ acción) es el lazo de entrenamiento; la derecha, el despliegue en el motor real.

4. Implementación

4.1. El lazo de entrenamiento (Python)

El entrenamiento corre íntegramente en Python. Cada entorno implementa la interfaz **gymnasium**: recibe la acción (v, ω) de ambos robots de campo, la integra en la física correspondiente y devuelve la observación de 52 entradas junto con la recompensa de la [table 2](#). Sobre rSoccer, el wrapper desarrollado hace cuatro cosas que resultaron decisivas: agrupa cuatro ticks de física de 40 Hz por decisión, de modo que la red decide a 10 Hz exactamente como en despliegue; perturba las poses con el ruido gaussiano medido y deriva las velocidades por diferencia finita, imitando la estadística de las estimaciones del filtro de Kalman del motor; replica la rampa de aceleración del ejecutor real antes de convertir a velocidades de rueda; y ejecuta un árbitro interno que cobra las faltas de área del reglamento y corta las jugadas estancadas. El rival y el arquero propio son heurísticos, para acotar el problema experimental.

PPO consume ocho copias del entorno en procesos paralelos. La política es la clase propia de dos torres descrita en la solución; un conjunto de callbacks guarda checkpoints cada 10^5 pasos, ejecuta evaluaciones deterministas periódicas conservando el mejor modelo, registra la recompensa por episodio y aplica la cota de $\log \sigma$ al cierre de cada rollout. Esta instrumentación, ausente en la primera corrida larga, fue la que permitió detectar el colapso de exploración y los óptimos locales que se analizan más adelante.

4.2. El despliegue (motor Rust)

Del modelo entrenado se exporta únicamente la media determinista del actor a ONNX. El motor del equipo carga ese archivo y lo evalúa cada seis ticks de su lazo de control de 60 Hz (decisión a 10 Hz). En cada decisión, el motor construye el mismo vector de 52 entradas a partir de su modelo de mundo —posiciones filtradas y velocidades estimadas por el EKF—, ejecuta la inferencia y traduce la salida al skill `DirectVel`: una rampa de aceleración suaviza el cambio de (v, ω) y la cinemática diferencial, con las constantes efectivas obtenidas por identificación de sistemas, convierte el resultado en velocidades de rueda que se envían a FIRASim por UDP (o a los robots por radio). Visión, *tracking*, planificación clásica y comunicación son componentes preexistentes del equipo: la red reemplaza únicamente la capa de decisión, y el resto del motor no requirió modificaciones para alojarla.

4.3. Decisiones de diseño

Control directo. Se eligió (v, ω) en lugar de objetivos de posición o skills de alto nivel. Esto impide que el planificador y el PID oculten una dinámica distinta entre entrenamiento y despliegue, aunque exige que la red aprenda locomoción y estrategia conjuntamente.

Rampa de aceleración. FIRASim reacciona de forma poco estable ante cambios instantáneos grandes. `DirectVel` limita la aceleración lineal a $1,2 \text{ m/s}^2$ al acelerar y 5 m/s^2 al frenar, y la angular a 30 rad/s^2 . El wrapper `rSoccer` replica esa rampa. El núcleo de la decisión cabe en tres líneas, mostradas solo como ejemplo:

Listing 1: Fragmento breve de la rampa replicada en entrenamiento.

```
a_lin = DV_LIN_ACCEL if abs(v_target) >= abs(v_cur) else DV_LIN_DECEL
v_cur += clip(v_target - v_cur, -a_lin * dt, a_lin * dt)
omega_cur += clip(omega_target - omega_cur, -DV_ANG_ACCEL * dt, DV_ANG_ACCEL * dt)
```

Identificación del ejecutor. Un barrido de giro puro mostró que FIRASim se comporta como si tuviera una separación efectiva $L = 0,085 \text{ m}$, estable en tres velocidades de rueda. El valor anterior $L = 0,05 \text{ m}$ ejecutaba aproximadamente un 59 % del giro solicitado. `rSoccer` se midió separadamente ($L \approx 0,077.6 \text{ m}$); usar la constante efectiva de cada simulador permite que el mismo comando abstracto (v, ω) tenga respuesta comparable.

Estabilidad de PPO. La primera corrida `rSoccer` usó una política SB3 estándar con $c_e = 0$, sin *target KL* ni cota explícita de $\log \sigma$. La versión corregida usa $c_e = 0,001$, *target KL*=0,05 y recorta $\log \sigma \in [-2; 0,7]$. Además, `Monitor` y `EvalCallback` registran recompensa y evaluación determinista.

Exportación compatible. Se exporta solamente la media del actor en ONNX opset 17. El exportador evita pesos externos porque el runtime `tract` espera un modelo autocontenido. La inferencia se realiza en CPU; el tamaño reducido de la red hace innecesaria una GPU en el robot.

5. Metodología experimental

5.1. Métricas y protocolo

Se reportan tres KPIs comprometidos desde la entrega parcial: tasa de goles, tiempo medio para completar el episodio y estabilidad entre episodios. Para partidos se agregan:

$$\text{net \%} = \text{goles a favor \%} - \text{goles en contra \%}.$$

Cada evaluación usa política determinista y semillas consecutivas. En el simulador propio se ejecutan 100 episodios por rival fijo. En rSoccer se ejecutaron 50 episodios contra el mismo oponente heurístico del entrenamiento. Los episodios duran como máximo 30 segundos.

También se inspeccionan métricas internas de PPO: varianza explicada del crítico, desviación estándar de la política, divergencia KL aproximada, entropía y pérdida de valor. Estas métricas no sustituyen el desempeño futbolístico, pero ayudan a distinguir un entorno difícil de una optimización inestable.

5.2. Experimentos

Tabla 3: Entrenamientos y finalidad experimental. Los resultados “en curso” no se usan para sostener las conclusiones cuantitativas.

Entrenamiento	Pasos	Tiempo	Finalidad
Fase 1, 1v0	1,5 M	12 min	Verificar aprendizaje desde cero
Currículo + self-play	~25 M	~4 h	Competencia frente a baselines fijos
rSoccer, intento 3	20,56 M	~22 h	Física realista; diagnóstico de colapso
rSoccer, gate 4a	8 M	~8 h	Ejecutor alineado; diagnóstico del reward
rSoccer, gate 4b	15 M planificados	en curso	PPO estable + reward de cercanía como nivel

El hardware principal fue un Intel Core i5-10500 de seis núcleos y doce hilos, sin GPU. Ocho entornos paralelos lograron aproximadamente 270–350 decisiones de agente por segundo. La simulación, no la red de 159 mil parámetros, fue el cuello de botella.

6. Resultados

6.1. Aprendizaje inicial

La [table 4](#) reproduce la comparación de la entrega parcial. El modelo final aún era débil, pero convirtió uno de cada veinte episodios y redujo el tiempo medio en 21 %, evidencia de aprendizaje desde cero. El aumento de desviación estándar se explica porque el modelo inicial siempre agotaba el tiempo, mientras el final mezcla éxitos tempranos y timeouts. Las curvas internas acompañan la progresión: la entropía baja (política menos uniforme), la σ de la gaussiana pasa de 0,96 a 0,75 y la varianza explicada del crítico se estabiliza en torno a 0,78.

Tabla 4: Fase 1, 100 episodios deterministas por checkpoint.

Métrica	100 k	1,5 M	Cambio
Tasa de goles	0,0 %	5,0 %	+5 pp
Recompensa media	-17,84	-8,40	+53 %
Desv. recompensa	0,41	8,29	aumenta
Tiempo medio	30,0 s	23,8 s	-21 %
Desv. tiempo	0,0 s	11,1 s	aumenta

6.2. Dominio en el simulador propio

Después del currículo y self-play, el mejor checkpoint anotó en 90 % de los episodios frente a *RuleBasedCoach* y recibió gol en 1 %; frente al rival agresivo *ball-chaser*, anotó 92 % y recibió 4 % (table 5). Este resultado valida que la arquitectura y la señal de aprendizaje pueden producir comportamiento efectivo dentro del entorno de entrenamiento.

Tabla 5: Evaluación del modelo self-play en el simulador propio ($n = 100$ por rival).

Oponente	Gol a favor	Gol en contra	Timeout	Neto
<i>RuleBasedCoach</i>	90 %	1 %	9 %	+89 pp
<i>Ball-chaser</i>	92 %	4 %	4 %	+88 pp

Sin embargo, el mismo modelo se movió de manera torpe en FIRASim y llegó a cometer autogoles. El contraste entre +89 puntos en entrenamiento y el fallo cualitativo en despliegue constituye evidencia de sobreajuste a la física simplificada, no de competencia general.

6.3. rSoccer: rendimiento y colapso

La corrida de 20,56 M de pasos se evaluó en el propio rSoccer antes de atribuir el problema a transferencia. El resultado fue 9 goles a favor, 3 en contra y 38 timeouts en 50 episodios: neto +12 pp, recompensa $2,09 \pm 6,93$ y 243 de 300 decisiones por episodio. Por tanto, el modelo era mediocre incluso en su entorno de entrenamiento. La figure 3 contrasta este resultado con el del simulador propio: son *modelos distintos*, cada uno evaluado dentro de *su propio* simulador, y el contraste (90 % frente a 18 % de goles) es precisamente la evidencia de los dos modos de falla analizados en la section 7.1.

TensorBoard mostró el mecanismo de fallo (figure 4): la desviación estándar cayó desde aproximadamente 0,86 hasta 0,045, la KL aproximada subió a 0,753 y la pérdida de entropía terminó en +6,75. Al mismo tiempo, la varianza explicada aumentó a 0,576. El crítico aprendió a predecir retornos, pero el actor se volvió casi determinista y realizó actualizaciones demasiado grandes. La política quedó atrapada en empujar la pelota sin definir.

El gate posterior de 8 M pasos, ya con ejecutor y observación alineados, reveló un segundo óptimo local: en 30 evaluaciones, la distancia mínima media a la pelota fue 0,51 m, 67 % de las jugadas terminó por *Free Ball* y la velocidad media fue 28 % del máximo. Este diagnóstico condujo al reward de cercanía como nivel. El reentrenamiento con los tres fixes estaba en ejecución al cierre; por rigor, no se le atribuyen resultados finales.

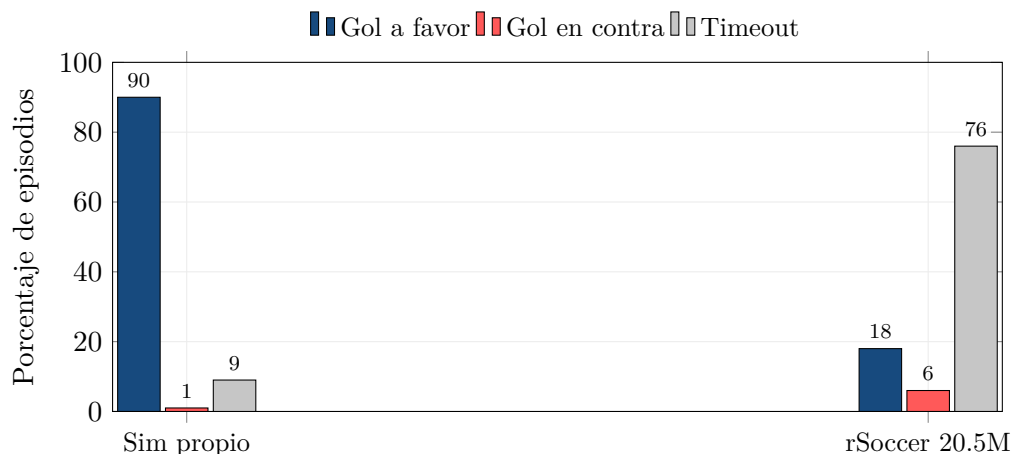


Figura 3: Desempeño determinista de dos modelos *distintos*, cada uno evaluado dentro de *su propio* simulador contra un oponente fijo: el modelo del simulador propio ($n = 100$, vs. RuleBasedCoach) domina, mientras que el modelo rSoccer ($n = 50$, vs. su rival heurístico) es mediocre en su propio entorno. Los datos frente a *ball-chaser* figuran en la [table 5](#).

7. Análisis crítico y comparación

7.1. Dos modos de falla diferentes

Los experimentos separan dos fenómenos que inicialmente parecían un solo *sim-to-sim gap*:

1. **Sobreajuste del simulador propio.** El modelo domina dentro de su entorno, pero falla en FIRASim. La explicación más probable es que explotó el contacto de pelota hecho a mano y otras regularidades específicas.
2. **Política rSoccer insuficiente e inestable.** Su neto de +12 y 76 % de timeout muestran que no había una política fuerte que transferir. La falta de entropía, límite KL y clip de $\log \sigma$ explica el colapso observado; el reward inicial explica la conducta pasiva posterior.

La distinción importa: domain randomization no puede rescatar por sí sola una política que no aprendió la tarea base, y más pasos no corrigen necesariamente una recompensa que premia el comportamiento equivocado.

7.2. La cooperación no se premió y, por lo tanto, no emergió

Una revisión cualitativa de los episodios evaluados y de los despliegues reveló un patrón consistente en los modelos entrenados: aunque una única red controla a ambos robots de campo, en la práctica juega uno solo. Un robot concentra toda la actividad —persigue, empuja y remata— mientras el segundo se desplaza sin propósito aparente, sin ocupar posiciones de pase ni de cobertura. No se observaron pases ni jugadas colectivas.

La auditoría de la recompensa explica el fenómeno como comportamiento *racional*, no como fallo de optimización. Primero, el término de cercanía considera solo al robot *más cercano* a la pelota: el segundo robot no recibe señal alguna por posicionarse. Segundo, ningún término premia el posicionamiento ofensivo (línea de pase) ni el defensivo (cobertura del arco propio); la versión inicial

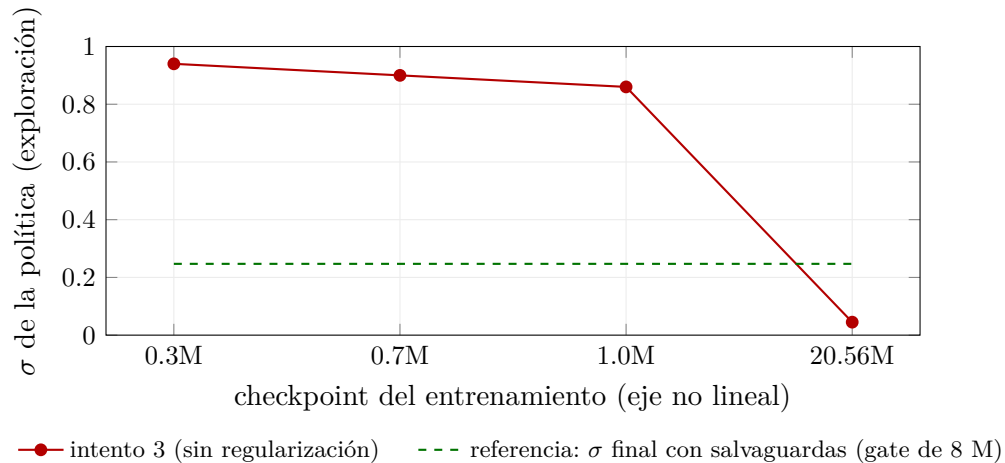


Figura 4: Colapso de exploración en la corrida rSoccer de 20,56 M de pasos: la desviación estándar de la política gaussiana se desploma hasta 0,045 (la red deja de explorar). La línea discontinua marca la σ final (0,247) del gate posterior, entrenado con bonus de entropía, *target KL* y cota de $\log \sigma$: con regularización, el colapso no se repite.

del término defensivo se eliminó por ser explotable y no fue reemplazada. Tercero, las faltas de área penalizan que dos robots coincidan en zonas calientes, de modo que alejarse de la jugada es la opción segura. Cuarto, la penalización por inactividad obliga a moverse, pero no a moverse *con propósito*: deambular en círculos es el óptimo local para un robot sin tarea asignada. En síntesis, la recompensa define implícitamente la división del trabajo del equipo, y la nuestra le asigna trabajo a un solo robot.

El resultado es consistente con la literatura. Brandão et al. [2] obtienen coordinación “emergente” solo bajo incentivos de equipo explícitos —términos de posicionamiento ofensivo y defensivo, y cercanía del *compañero* más próximo a la pelota— combinados con presión de self-play, mientras que los benchmarks multiagente de rSoccer reportan agentes que no colaboran e incluso se obstaculizan mutuamente [3]. La lección de diseño es directa: en RL multiagente la cooperación no es un subproducto gratuito de compartir el objetivo, sino que requiere que el robot sin pelota tenga valor esperado por posicionarse. La corrección propuesta combina términos de posicionamiento no explotables, presión de self-play (un equipo que juega con un solo robot es explotable por cualquier rival competente) y una métrica de participación por robot que permita verificar la cooperación cuantitativamente, no solo por inspección visual.

7.3. Brechas entrenamiento-despliegue

La auditoría encontró cuatro desajustes principales:

La evaluación en FIRASim fue cualitativa porque el motor no dispone aún de un marcador automático. Esta es una limitación importante: permite detectar torpeza o autogoles, pero no estimar intervalos de confianza ni comparar checkpoints con el mismo rigor que dentro de los simuladores.

7.4. Comparación con trabajos relacionados

Brandão et al. [2] integran control y estrategia en VSSS mediante self-play y reportan tasas de

Tabla 6: Brechas identificadas y mitigaciones implementadas.

Brecha	Efecto esperado	Mitigación
Física de contacto	Política explota rebotes o fricción particulares	Migración a rSoccer; pendiente randomización y validación contra FIRASim
Velocidad perfecta vs. estimada	Ruido y retardo alteran 28 componentes de velocidad	Ruido de pose y diferencia finita a 10 Hz
Ejecutor distinto	(v, ω) solicitado no coincide con el movimiento	Rampa DirectVel replicada en rSoccer
Geometría de ruedas	Giro ejecutado era 59 % del solicitado	System-ID: $L_{FIRASim} = 0,085 \text{ m}$ y $L_{rSoccer} = 0,077.6 \text{ m}$

victoria de hasta 94 % contra estrategias heurísticas; contra otra heurística de referencia reportan 84 %. Su entrenamiento utiliza cientos de millones de transiciones, self-play y ruido medido. En contraste, nuestra corrida rSoccer principal usa 20,56 M, rival fijo y, en su primera versión, sin domain randomization. La diferencia de escala y robustez vuelve esperable la brecha de desempeño.

rSoccer fue concebido como framework para estudiar RL y sim-to-real en VSSS/SSL y sus benchmarks también exponen limitaciones de los algoritmos en escenarios complejos [3]. Nuestro resultado confirma que sustituir el simulador no basta: el entorno, la frecuencia de control, la acción, la observación y la recompensa deben alinearse como sistema. La randomización de dominio ofrece una vía para entrenar frente a una distribución de dinámicas, en lugar de una única física nominal [6].

VSSS-RL logró competencia física mediante una capa de adaptación de bajo nivel [4]; esto respalda la importancia que aquí tuvo calibrar el ejecutor. Coach-RL, en cambio, aprende asignación de roles sobre controladores clásicos [5]. Esa alternativa reduce la carga de aprendizaje y facilita transferencia, pero limita la posibilidad de descubrir control y estrategia conjuntamente. El aprendizaje de habilidades individuales en la Small Size League [11, 12] ocupa un nivel de abstracción intermedio entre el control de bajo nivel y la asignación de roles. Nuestro enfoque de bajo nivel es más ambicioso y, por lo mismo, más sensible al modelo físico.

8. Conclusiones y trabajo futuro

Se implementó un pipeline funcional de extremo a extremo: generación de experiencia, entrenamiento PPO, exportación ONNX, inferencia en Rust y control de FIRASim a 10 Hz. El contrato de 52 entradas y la acción (v, ω) permiten reemplazar la capa de decisión sin reescribir visión, tracking ni comunicación.

La política aprendió comportamiento efectivo dentro del simulador propio y superó ampliamente al baseline basado en reglas (+89 pp netos). No obstante, ese desempeño no transfirió. La corrida de física realista tampoco resolvió el problema: obtuvo +12 pp y 76 % de timeout dentro de rSoccer, junto con señales claras de colapso de exploración. En consecuencia, no se sostiene que el sistema sea competitivo; sí se demuestra que aprende, se despliega y que sus fallos pueden medirse y atribuirse a componentes concretos.

El análisis produjo cambios verificables en el código: regularización de PPO, monitoreo, reward orientado al remate y a la cercanía sostenida, identificación del wheelbase, rampa compartida y ruido de observación. Estos resultados negativos son informativos porque transforman una “transferencia torpe” difusa en una hoja de ruta experimental.

El trabajo futuro se prioriza así:

1. terminar y evaluar el reentrenamiento rSoccer corregido con al menos 100 episodios y múltiples semillas;
2. automatizar el marcador en FIRASim y evaluar allí cada checkpoint contra RuleBasedCoach;
3. incorporar self-play con snapshots y currículo en el stack rSoccer;
4. agregar incentivos de cooperación y defensa (posicionamiento ofensivo y defensivo no explotables, al estilo de 2) y medir la participación de ambos robots (toques y distancia a la pelota por robot);
5. aplicar domain randomization sobre fricción, masa, latencia, ruido y parámetros del ejecutor;
6. aumentar la escala hacia 50–100 M pasos solo después de validar reward y estabilidad;
7. entrenar o coadaptar el arquero y espejar observaciones/acciones para ambos colores de equipo.

Disponibilidad y reproducibilidad

El código, la configuración, los scripts de entrenamiento y evaluación, y las fuentes de este informe están disponibles en <https://github.com/Sysmic-Robotics/VSSS-Software>, rama `feature/rl-fase1-red-neuronal`. Las instrucciones de instalación y ejecución se encuentran en `README.md` y `training/README.md`. No se almacenan datasets pesados porque la experiencia se genera en línea.

Referencias

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint* arXiv:1707.06347, 2017.
- [2] B. Brandão, T. W. de Lima, A. Soares, L. Melo, and M. R. O. A. Maximo, “Multiagent Reinforcement Learning for Strategic Decision Making and Control in Robotic Soccer Through Self-Play,” *IEEE Access*, vol. 10, pp. 72628–72642, 2022.
- [3] F. B. Martins, M. G. Machado, H. F. Bassani, P. H. M. Braga, and E. S. Barros, “rSoccer: A Framework for Studying Reinforcement Learning in Small and Very Small Size Robot Soccer,” *arXiv preprint* arXiv:2106.12895, 2021. github.com/robocin/rSoccer.
- [4] H. F. Bassani *et al.*, “A Framework for Studying Reinforcement Learning and Sim-to-Real in Robot Soccer,” *arXiv preprint* arXiv:2008.12624, 2020.
- [5] C. Pena, H. Machado, *et al.*, “An Analysis of Reinforcement Learning Applied to Coach Task in IEEE Very Small Size Soccer,” *arXiv preprint* arXiv:2011.11785, 2020.
- [6] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.
- [7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [8] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *arXiv preprint* arXiv:1412.6980, 2014.
- [9] C. M. Bishop, *Deep Learning: Foundations and Concepts*. Springer, 2024.
- [10] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [11] D. Schwab, Y. Zhu, and M. Veloso, “Learning Skills for Small Size League RoboCup,” in *RoboCup 2018: Robot World Cup XXII*. Springer, 2018.
- [12] M. Yoon, “Developing basic soccer skills using reinforcement learning for the RoboCup Small Size League,” M.Sc. Thesis, Stellenbosch University, 2015.